

爆速プロトタイプ構築術 | Google AI Studioではじめる Vibe Coding

20分で、動く。 その先を、決める。

AIにコードを書かせて、人に渡す前に何を確かめるかまで

麗澤大学 工学部 小塩 篤史



今日のゴール: 3つのことができるようになる



20分で動くアプリを作る

日本語で頼むだけで、ボタンで記録が増えるアプリを作り、スマホで開く



AIへの頼み方の型を使う

「プロンプト」を「仕事の依頼」に変える4項目と3つの鉄則



動いた先を言葉にする

保存・公開・確認・責任。人に渡す前に何が足りないかを挙げられる

定義 5分

道具 6分

ライブデモ 14分

頼み方 7分

その先 5分

演習 15分

講義 約40分 + 演習 約15分(残りは質疑・予備)

PART A

Vibe Codingとは何か

速さの正体と、引き換えに後回しにしているもの

Vibe Coding = 日本語で頼み、結果を見て進めるスタイル



Andrej Karpathy が名付けた(2025年2月、Xへの投稿)

- 雰囲気(vibe)に身をまかせ、コードの存在すら忘れて進める
- AIが提案する変更を、細かく読まずに「すべて受け入れる」
- エラーが出たら、メッセージをそのままAIに貼り付ける
- コードが自分の理解を超えて育っても、先へ進む

出典の要旨。教科書 第1章 1.1

手に入るもの

速い・始めやすい・対話できる

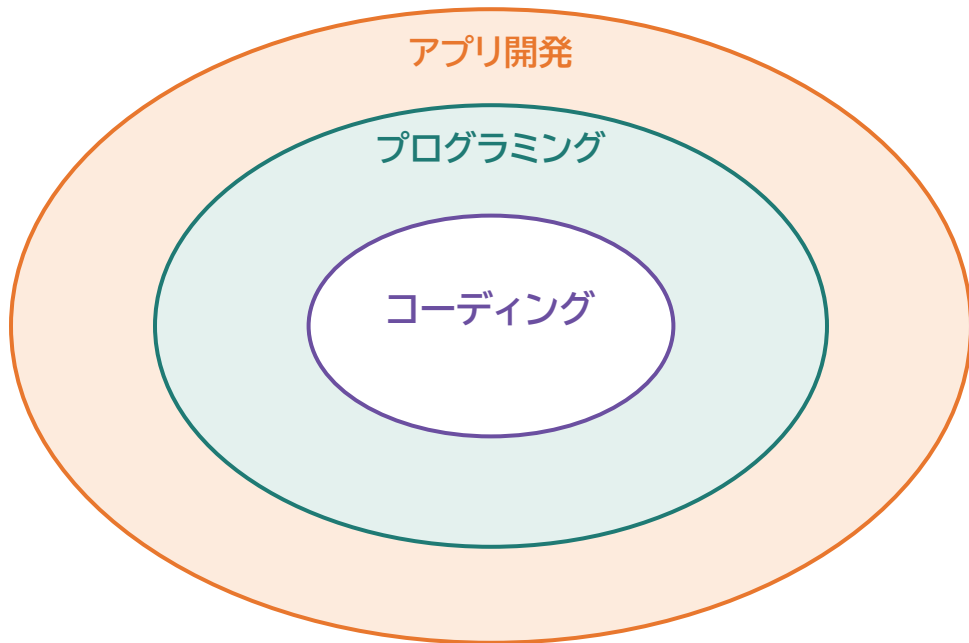
後回しにしているもの

コードの理解・変更内容の確認・検証
と対応準備

速さと引き換えに、理解と確認を
後回しにする。それがこのスタイル
の正直な設計

どこまでAIに任せる？

コーディング ⊂ プログラミング ⊂ アプリ開発



コーディング

決まった内容をコードに書く(清書)

AIが強く助ける

プログラミング

手順を考え、組み立て、動かして確かめる

AIがかなり助ける

アプリ開発

誰の困りごとを解くか決め、確かめ、届け、使われ続ける状態を保つ

人の判断が増える

今日のデモで触れるのは、いちばん内側だけ

何のために作る？ 呼び方は「完成度」ではなく「目的」



デモ 見せる

動く場面を見せればよい



プロトタイプ 触ってもらって学ぶ

迷う場所・要らない機能を見つける



PoC 実現できるかを確かめる

「この技術で可能か」の一点を証明



MVP 利用者の仮説を確かめる

最小限の製品で「本当に助かるか」を学ぶ

本番＝実際に誰かが使っている「状態」の名前。今日作るのは「デモ」

PART B

道具: Google AI Studio「Build」

インストール不要。ブラウザだけで、チャットとプレビューがそろふ

なぜ AI Studio の Build から始めるのか

0

インストール

ブラウザとGoogleアカウントだけ。学校PCでも始めやすい

無料

使える範囲で

AI Studioの利用自体は無料(1日の利用上限あり)

2つ

最初に使う道具

チャット(頼む)とプレビュー(確かめる)が1画面にそろう

中身は本物のWebアプリ: 既定でReact(画面の部品を組み立てる仕組み) + Node.js(サーバー側でJavaScriptを動かす環境)のプロジェクトが生成される。

次の一步: ファイル・ターミナル・Gitまで使うときは Antigravity へ (BuildからGitHub連携・書き出しができる)

はじめる前の4つの確認(細かい設定)



1. 開く

aistudio.google.com → 左メニュー「Build」

ブラウザの拡張機能が原因でエラー(403)が出ることも。出たらシークレットウィンドウで



2. ログイン

大学のGoogleアカウントで(18歳以上)

「アクセス権がありません」と出たら大学の管理者設定。生年月日の確認を求められることもある



3. 上限を知る

無料でも1日の上限がある(モデルごと)

「Quota exceeded(上限超過)」が出たら、別のモデルに切り替えるか翌日に



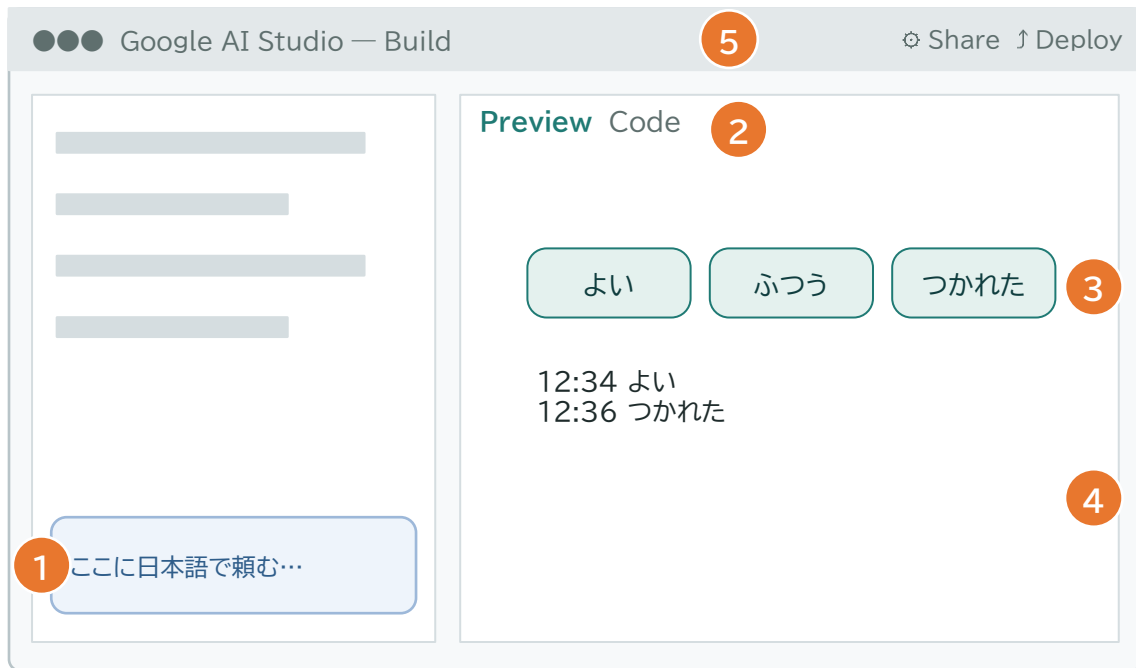
4. 入れないもの

個人情報・パスワード・APIキー・授業の機密

無料の利用では、入力が製品改善に使われ、人が読む場合がある

画面の名前や配置はアップデートでよく変わる。見つからないときは「この画面でプレビューを全画面にする方法は？」とAIに聞いてよい

Build画面の地図:どこで頼み、どこで確かめるか



- 1 チャット(プロンプト欄)**
日本語で頼む。音声入力や「I'm Feeling Lucky」(おまかせ案)も
- 2 Code タブ**
AIが作ったファイルの一覧と中身。直接編集もできる
- 3 Preview(プレビュー)**
作ったアプリがその場で動く。全画面・スマホ幅の表示も
- 4 Annotation mode**
プレビューの一部を囲んで「ここを直して」と指示できる
- 5 設定・共有・公開**
モデル選択、Secrets(秘密の値の金庫)、Share、GitHub、Deploy

AI開発環境の「6つの道具」は、AI Studioのどこにある？

道具	何に使うか	AI Studio Build では	初出
AIとのチャット	作る・直す・質問する	左のチャット欄	序章
プレビュー	作ったアプリを動かして確かめる	右のPreview(全画面・スマホ幅)	序章
ファイル	AIが書いたコードを開いて読む	Codeタブのファイル一覧	第6章
ターミナル	コマンドで直接指示する	なし → Antigravity等で	第10章
バージョン管理	セーブ(コミット)と戻る	GitHub連携／チェックポイント(要確認)	第10章
公開	他の人がURLで使える状態にする	Share(共有)／Deploy(Cloud Run)	第14章

今日使うのは上の2つだけ。残りの4つは「場所をメモしておく」で十分

PART C LIVE DEMO

20分でアプリを作ってみる

題材: 世界最小の日記「気分を記録する3つのボタン」(教科書 序章)

DEMO① 日本語で頼むだけで、アプリができる

あなた → AI(Buildのプロンプト欄にそのまま貼る)

気分を記録する練習用アプリを作ってください。画面に「よい」「ふつう」「つかれた」の3つのボタンを置き、押すと時刻付きの記録が1行ずつ増えるようにします。この演習では記録を画面のメモリだけに置き、再読み込みすると消える仕様にしてください。ログインや外部送信は不要です。

教科書 序章 0.2 の依頼文

あなたはまだ、コードを1行も書いていない



クリア条件

- 3つのボタンが並ぶ
- 押すと「12:34 よい」のように記録が増える
- 何回か押して積み上がる

AIの返事・見た目は毎回違う。本と同じでなくてよい。頼んだ動きができていれば合格

DEMO② 言葉で変える。具体的に頼むか、曖昧に頼むか

具体的な頼み方

「つかれた」のボタンだけ紫色にしてください。あと、記録の文章を「17:03 つかれた」のように短くしてください。

→ 結果を見比べやすい

曖昧な頼み方

全体を、もっとやさしい雰囲気にしてください。

→ 色を淡く？角を丸く？ 人によって解釈が分かれる

曖昧な部分は、AIが「仮定」で埋める。違っていたら言い直せばよい。変えたら、色だけでなく「記録が増える」ことも確かめる

DEMO③ スマホで開いて、隣の人に押してもらおう



1. 共有用のURLを出す

右上の「Share(共有)」でアプリのリンクを作る



2. QRコードにする

「このURLのQRコードを作って」とAIに頼むのも手



3. スマホで開く

開けないときは、PCでブラウザの幅を狭くして仮確認

localhost / 127.0.0.1 は「その端末自身」を指す住所。PCのURLをスマホに渡してもつながらない

アプリは最初は非公開。共有した相手はコードも見られ、複製(fork)できる。相手が使ったAIの分は自分の利用枠に数えられることがある

ボス出現:再読み込みしたら、記録が消えた

メモリだけに置いた記録

10:12 よい
10:18 ふつう
10:26 よい



再読み込み後

記録 0件

黒板のメモ

メモリ＝画面が開いているあいだだけの置き場所

ノート

保存＝あとで読み戻せる場所へ書くこと(永続化)

「画面に表示されている」と「保存されている」は別。壊れたのではなく、まだ保存の仕組みがないだけ

サークル20人で毎日使える？ とAIに聞いてみる

あなた → AI

このアプリを、私のサークルのメンバー20人で毎日の気分を共有するボードとして本番で使えますか。問題があれば、遠慮なく全部挙げてください。

AIの返事の例

-  記録がどこにも保存されない
-  URLは一時的。環境を閉じると開けない
-  20人の誰の記録か区別できない
-  壊れたとき、原因を調べて直す手順がない
-  個人的な記録を預かるのに、安全を確認していない

「画面を作る依頼」と「20人が毎日使えるかを点検する依頼」では、確認する範囲が違う

用語① クライアント／サーバー、リクエスト／レスポンス



最初に立てる問い:「このアプリでは、どれがサーバー？ 止まると何が起きる？」

用語② フロントエンド／バックエンドと「保存」の3段階

フロントエンド＝画面側(客席で起きること) バックエンド＝向こう側の処理(厨房で起きること)

置き場所	日常語で言うと	再読み込み	別の端末・他の人
メモリ(変数)	黒板のメモ	消える	見えない
ローカルストレージ	そのブラウザ専用のノート	残る	見えない(端末間で共有しない)
データベース(サーバー側)	みんなで使える頑丈な専用ノート	残る	見える(権限の設計が必要)

「保存して」とだけ頼むと、AIはどれかを選ぶ。「どこに保存しましたか？」と必ず聞き返す

教科書 第5章 5.3、第8章(用語ミニ辞典より)

用語③ 開発者ツール=ブラウザに隠れている観察窓

開き方: プレビューを別タブで開き、F12 (Macは $\text{⌘}+I$) またはブラウザのメニュー → 「デベロッパーツール」



Console(コンソール)

エラーが赤字で出る場所。「画面が真っ白」のときはまずここ

エラー文は全文コピーしてAIへ(秘密は伏せる)



Network(ネットワーク)

リクエストとレスポンスの往復が1行ずつ並ぶ。200 = 成功

「ボタンを押したとき、何か送られたか」を見る

気分アプリで「記録」ボタンを押しても、Networkの行は増えない = 通信していない。通信していないことと、保存していないことは別

PART D

「プロンプト」を「仕事の依頼」に変える

うまい言い回しより、AIが知っている前提と、頼む大きさ

AI開発ブリーフ:依頼に付ける4項目

プロンプト=AIへの依頼文 コンテキスト=AIに渡す前提情報。結果を分けるのは言い回しより前提

1 目的

なぜやるのか(困りごとに戻る)

なぜ

2 利用者

誰のためか

誰

3 制約

守ってほしい条件。してほしいことより先に「変えないでほしいこと」

変えない

4 完成条件

どうなったら完了か。数字で書けると強い

できた？

例:気分アプリに保存を足す

【目的】再読み込みしても、その日の気分を見返せるようにしたい

【利用者】自分ひとり(このPCのブラウザだけ)

【制約】ボタン3つと見た目は変えない。外部送信・ログインはしない

【完成条件】記録を3件→再読み込み→3件のまま残る

まず計画だけ出してください。実装はまだしないでください。

壊れの連鎖を止める3つの鉄則



まず計画を出させる

「まず計画だけ出してください。実装はまだしないでください」

作る前なら、ずれを言葉の段階で直せる



1回の依頼=1つの変化

「取り消しを直して、ついでに集計も、あと色も」は3つの変化

完成条件が1~2行で書けないタスクは、まだ大きすぎる



仮定と不明点を挙げさせる

「実装の前に、あなたが置く仮定と、決まっていないことを列挙して」

答えた内容は仕様書に書き戻す

直らないときは同じ指示を繰り返さない: ①言い方を変える(事実を書く) ②小さくする ③戻す

エラーが出たら／AIに自分でテストさせる



エラーの伝え方

- 何をしたら、何が起きたか(操作と結果)
- エラー文は要約せず、全文を貼る
- 貼る前に、APIキー・パスワード・個人情報を伏せる(例:apiKey=[伏せた])
- 自動修正に任せたら「何が原因だったか」も聞く



役割を分ける

実装役 ブリーフ通りに作る

テスト役 壊すつもりで確かめる(検算・意地悪な入力)

レビュー役 仕様と比べ、漏れと余計な変更を探す

AIの「確認しました」は証拠ではない。実行した方法と結果を出させ、自分の画面でも試す

PART E

「動いた」を「使える」に育てる

デモから人に渡せるアプリまでの、5つの壁

5つの壁を裏返すと、「開発できる」の5条件になる

壁	使う地図	手に入る力	
伝わらない	価値の地図	説明できる	誰の何を解決し、どういう仕組みか言える
中が見えない	構造の地図	変更できる	直したい場所を狙って変えられる
変更が怖い	変更の地図	戻せる	壊れても、壊れる前に帰れる
確かめられない	品質の地図	確かめられる	「正しく動く」を根拠で示せる
渡せない	運用の地図	渡せる	自分がいなくても使い続けられる

今日のアプリは「説明できる」の入口に立ったところ。1つずつ増やせば、次の変更を安心して試せる

AIに任せられる仕事、任せきれない仕事

AIに任せられる仕事(作業)	任せきれない仕事(判断と責任)
コードを書く	何を作るかを決める
エラーの原因の候補を挙げる	どの直し方が自分の現場に正しいか決める
問題点や心配ごとを列挙する	どれを重く見て、どれを捨てるか決める
公開の手順を教える	公開してよいと判断する
直し方を提案し続ける	壊れたとき、最後に対応する人になる

コード生成はAIに任せられる。

だが、何を作るか、正しいか、公開してよいか、誰が守るかは任せきれない。

人に渡す前の5つのチェック



個人情報

他人の名前・連絡先・健康などを、AIにもアプリにも入れていないか



秘密の値

APIキーやパスワードをコードに直書きしていないか(Secretsに置く)



URLの範囲

誰が・いつまで開けるか。共有相手はコードも見られる



利用枠

相手が使った分のAI利用は誰の枠か。上限に達したらどうなるか



使ってもらう相手

学外の人には目的を説明し同意を得る。18歳以上に限る

演習: デモと同じアプリを、自分の手で作る(約15分)

1

5分

作る

DEMO①の依頼文を
Buildに貼って送る

3つのボタンを押して、記
録が増えればOK

2

3分

変える

見た目を1か所、日本語
で頼んで変える

例:「つかれた」のボタンを
紫にして

3

2分

見せる

隣の人にボタンを押し
てもらう

スマホでも開けたら、なお
よし

4

3分

書き出す

「分からなかったこと」
を2つ以上書く

例:再読み込みで消えた
のはなぜ?

出すもの(2つ)

1. アプリの画面の写真(スクリーンショット)
2. 「分からなかったことリスト」(2つ以上)

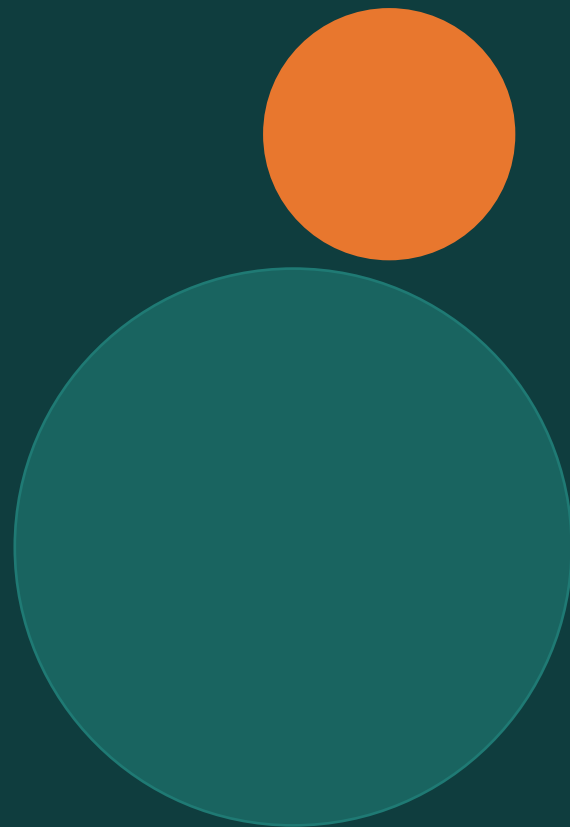
早く終わった人: ボタンを4つにし、4つめは自
分の言葉(「ねむい」など)に

困ったら: 新しいチャットで最初からやり直し
てよい

まとめ

コードはAIに。 判断はあなたに。

- ・ 20分で「動くデモ」は作れる。でも、保存・公開・確認・責任はまだ空白
- ・ 頼み方は、目的・利用者・制約・完成条件+「まず計画だけ」
- ・ 「動いた」のあとに「ここを確かめよう」と一歩進めることが、開発



APPENDIX

付録：細かい設定・用語・困ったとき

配布用。講義では必要に応じて見せる

付録1 Geminiを呼ぶときの設定(Playground)

AI Studioの「Chat(Playground)」で試し、右側の Run settings(調整アイコン)で変える。決まったら「Get code」でコードにできる

設定	何が変わるか	初心者へのひとこと
モデル	賢さ・速さ・無料枠。無料で使えるのは主にFlash系	迷ったら既定のFlash。Pro系のプレビューは有料のみ
System instructions	AIの役割・口調・守るルールを最初に渡す	アプリの「人格と禁止事項」をここに書く
Thinking level	答える前に考える量(minimal/low/medium/high)	上げると丁寧だが遅く、トークンも増える
Temperature	答えのばらつき	最新のFlashは変更しない推奨。古い解説に注意
Tools	構造化出力(JSON)、関数呼び出し、コード実行、Google検索での裏付け	アプリでデータを扱うなら構造化出力が便利
Safety settings	有害な出力をどこまで止めるか	既定のままでよい
Get code	今の設定をコード(Python/JavaScript等)にする	APIキーはコードに直書きしない

付録2 Buildの設定・共有・書き出し

項目	何ができるか	注意
Build agentのモデル	アプリを作るAIのモデルを選ぶ	上限に達したら別モデルに切り替えると続けられることがある
Secrets	APIキーなど秘密の値を、サーバー側に保管	GeminiのキーはBuildが自動設定。画面側には出ない
Share(共有)	特定の人にアプリを見せる・編集させる	既定は非公開。相手はコードを見てfork(複製)できる
GitHub	コードをGitHubと同期する	第10章の「変更履歴」へのつながり
ダウンロード(ZIP)	コード一式を手元に持ち出す	外で動かすにはGEMINI_API_KEYを自分で設定
Deploy(Cloud Run)	インターネットに公開する	料金が発生する場合あり。授業では講師デモのみ推奨
Firebase連携	データベース(Firestore)やログイン(Auth)を自動で構成	承認を求められたら、何が作られるか読んでから

付録3 用語ミニ辞典①:作る・頼む

用語	日常語で言うと	用語	日常語で言うと
Vibe Coding	AIとの対話を中心に作るスタイル	プロンプト	AIへの依頼文
デモ	見せるための試作	コンテキスト	AIに渡す前提情報
プロトタイプ	試して学ぶための試作	制約	守ってほしい条件
PoC	実現できるかの確認	完成条件	どうなったら完了か
MVP	仮説を確かめる最小限の製品	タスク分割	大きな仕事を小さく割る
本番アプリ	実際に使われているアプリ	仕様	作るものの決めごと(注文書)
プレビュー	作ったものがその場で映る画面	受け入れ条件	「確認できたら完成」の一覧

付録4 用語ミニ辞典②:仕組み・公開

用語	日常語で言うと	用語	日常語で言うと
クライアント	頼む側(あなたの手元)	API	アプリ同士の注文窓口
サーバー	頼まれて処理を返す側	APIキー	窓口の会員証(他人に見せない)
フロントエンド	画面側	レートリミット/クォータ	回数制限/利用量の上限
バックエンド	向こう側の処理	永続化	あとで読み戻せる場所へ保存
リクエスト/レスポンス	注文/返事	ローカルストレージ	そのブラウザ専用の保存領域
ローカル/クラウド	自分の環境の中/借り物のコンピューター群	データベース	みんなで使える頑丈な専用ノート
開発者ツール	ブラウザに隠れている観察窓	デプロイ	店舗への引っ越し(公開)

付録5 つまずき別チートシート

症状	まず見る場所	AIへの頼み方(例)
画面が真っ白	開発者ツールのConsole(赤いエラー)	「画面が真っ白です。コンソールのエラーを貼ります:(全文)。原因の候補を挙げて」
ボタンを押しても何も起きない	ConsoleとNetwork(押した瞬間に何か送られたか)	「〇〇ボタンが反応しません。押したときのコンソールとネットワークの記録:(貼る)」
スマホで表示が崩れる	プレビューのスマホ幅表示／実機	「画面幅が狭いとき〇〇が重なります。スマホ表示を直して」
Quota exceeded／上限の表示	Buildのモデル設定	別のモデルに切り替える。だめなら時間をおく
直すたびに別の場所が壊れる	直前の変更(何を頼んだか)	「1回で1つだけ変えて。計算と表示の動きは変えないで」→だめなら戻す

付録6 出典と注意

- 教材:『Vibe Codingからはじめるアプリ開発の教科書』教科書改訂版(2026-09-28)序章、第1・4・5章、付録A～C
- Google AI for Developers「Build mode」(2026-08-20更新)／「AI Studio quickstart」(2026-07-30)／「Pricing」(2026-10-01)／「Rate limits」(2026-09-02)／「Thinking」(2026-09-25)
- Gemini API 追加利用規約(2026-03-23改定)、「Google AI Studio for Workspace users」(2026-04-28)
- Google AI Developers Forum「Daily usage limits for models in AI Studio」(2026-09-29)
- Google Blog「Full-stack vibe coding in Google AI Studio」(2026-03-18)、Firebase Blog(2026-03-19, 2026-05-19)、Google Cloud Blog(2026-05-22)
- Andrej Karpathy の投稿(X、2025年2月)

AI Studioの画面・無料枠・規約は頻繁に変わります。この資料の内容は2026年10月6日時点の調査に基づきます。本資料はAIの支援で作成しており、講義前に講師が実際の画面と最新の公式情報で確認してください。